ABE5009 Final Project Report

Environmental Control Using an Embedded Controller for a Small Chamber

- Name(s): Arthur Borges Bringel Machado, Sparsh Kapoor, Adam
- Course: ABE5009 Control Systems in SmartAg
- Date: May 3rd, 2026

1. Objective

This project developed an embedded environmental control system for a small insulated chamber using a Raspberry Pi. The system was designed to measure chamber temperature and relative humidity and automatically actuate heating, ventilation, and humidity-related control actions without continuous human intervention.

The project also compared controller behavior using conventional PID control and fuzzy-PID control for temperature regulation, while using threshold-based humidity management and ventilation logic for relative humidity control.

2. Problem Statement

The goal of the final project was to build and evaluate an embedded environmental control system for a chamber capable of heating, ventilation, and humidity manipulation. The system had to integrate sensor measurement, actuator control, closed-loop software implementation, and experimental data logging at approximately `1 Hz`.

In addition to building the physical system, the project required comparison between a standard PID controller and an additional control strategy. In this implementation, the comparison was performed between conventional PID control and fuzzy-PID control for temperature regulation, while humidity was managed through threshold-based ventilation logic under a persistent humidifier disturbance.

3. System Overview

The system operates as a closed-loop chamber controller built around a Raspberry Pi, a DHT11 temperature and humidity sensor, a heater, and a ventilation fan. At each sample, the sensor measures the chamber temperature and relative humidity. The software then calculates a temperature-control effort using either PID or fuzzy-PID logic and combines that result with humidity threshold logic to determine the final actuator commands.

The heater is driven by time-proportioned switching so that a bounded controller output can be translated into an effective duty cycle over a fixed time window. The fan is used as a binary actuator for both cooling and dehumidification.

During the experiments, all measured and commanded signals were logged to CSV and could also be displayed in real time for monitoring.

4. Hardware and Materials

4.1 Materials

The main hardware used in the project is summarized below.

Item	Quantity	Purpose
Raspberry Pi 4	1	Embedded controller
DHT11 sensor	1	Temperature and RH measurement
Heating pad	1	Temperature increase
Fan	1	Ventilation / cooling / dehumidification
MOSFET	2	Actuator switching
Power supply	2	Power for Pi and actuators
Chamber enclosure	1	Controlled environment
Humidifier	1	Humidity addition
Jumper wires / breadboard / connectors	1 set	Wiring

Table 1: Main hardware components used in the environmental chamber system.

4.2 Chamber Construction

The environmental chamber was constructed from a foam cooler box, which provided a lightweight and inexpensive insulated enclosure. The foam cooler reduced direct influence from room conditions and helped retain thermal energy within the chamber during testing.

The heater, fan, sensor, and humidifier were placed inside or connected to the chamber so that the Raspberry Pi could monitor internal conditions and actuate heating and ventilation. The foam construction made it practical to route wires and mount small components while still preserving basic insulation. The chamber therefore served as a simple but effective experimental platform for evaluating closed-loop environmental control behavior.

In this project, the humidifier was present in the chamber but was not electronically controllable by the Raspberry Pi. The available humidifier hardware required a manual button press on its onboard circuit before it would begin operating, even when external power was already applied. Because of this behavior, the humidifier could not be reliably switched by the MOSFET interface and was instead treated as a constant disturbance source during experiments.

5. Wiring and Electrical Integration

5.1 Wiring Description

The DHT11 sensor data line was connected to Raspberry Pi GPIO BCM `4` for chamber temperature and relative humidity measurement. The heater switching stage was connected to GPIO BCM `18`, and the fan switching stage was connected to GPIO BCM `17`. Both actuators were driven through external switching hardware so that the low-power GPIO outputs could control the higher-power heater and fan loads safely.

All devices shared a common electrical reference, and the actuator power path was separated from the logic-level GPIO control path through the switching stage. This allowed the Raspberry Pi to command heating and ventilation without directly sourcing the actuator current.

5.2 Wiring Diagram

The wiring diagram for the hardware implementation is shown below.

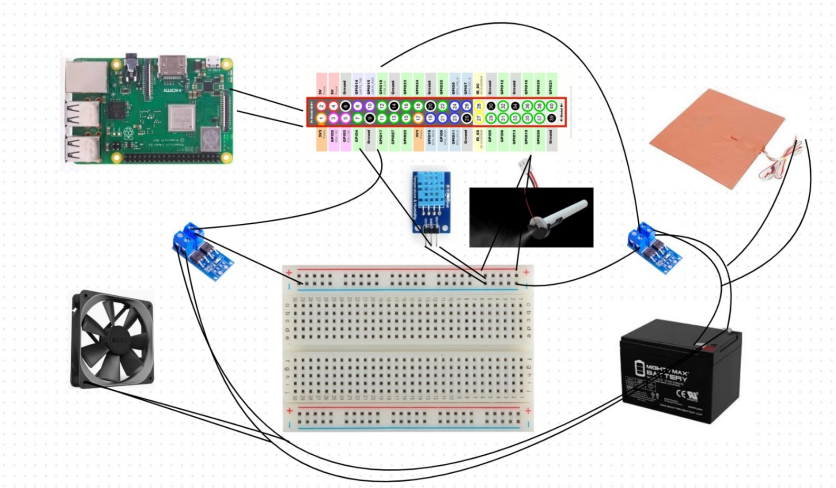

Figure 1: Wiring diagram showing the Raspberry Pi, sensor, and actuator switching connections.

6. Embedded Controller Implementation

6.1 Software Files

The embedded software was organized into modular files so that sensing, control, and actuator logic were separated clearly:

- main.py: program entry point and argument parsing
- climate_control.py: main control loop, CSV logging, plotting

- controllers.py: PID and fuzzy-PID control logic
- heater.py: actuator abstractions and time-proportioned heater control
- fan.py: binary fan actuator logic
- thsensor.py: DHT11 interface

6.2 Closed-Loop Sampling and Signal Flow

The control loop runs in climate_control.py. At each sample:

1. The DHT11 sensor provides temperature and relative humidity.
2. The loop computes `dt` from the current timestamp and previous timestamp.
3. The temperature controller generates a signed effort in the range `[-1, 1]`.
4. The humidity controller decides whether ventilation should be forced on.
5. The climate logic converts those decisions into heater and fan actuator commands.
6. The heater output is time-proportioned over a fixed window, while the fan is binary.
7. The loop logs `timestamp`, `temperature`, `relative_humidity`, `heater_command`, and `fan_command`.

The signed temperature control effort is interpreted as:

- positive effort: heating demand
- negative effort: cooling demand

This allows one controller output to be mapped into two physical actions:

- heater actuation for positive effort
- fan-based cooling for negative effort

6.3 PID Temperature Controller

The standard PID controller is implemented in controllers.py. The controller computes the temperature error as:

$$e(k) = T_{sp}(k) - T(k)$$

where:

- T_{sp} is the temperature setpoint
- T is the measured chamber temperature

The integral term is updated as:

$$I(k) = \text{clamp}\left(I(k-1) + e(k)\Delta t,\ -I_{\max},\ I_{\max}\right)$$

The derivative term is:

$$D(k) = 0 \text{ for the first sample}$$

$$D(k) = \frac{e(k) - e(k-1)}{\Delta t} \text{ for all later samples}$$

The controller output is:

$$u(k) = \text{clamp}\left(K_p e(k) + K_i I(k) + K_d D(k),\ -u_{\max},\ u_{\max}\right)$$

In the current code, the default PID parameters are:

- $K_p = 0.35$
- $K_i = 0.03$
- $K_d = 0.10$
- $u_{\max} = 1.0$
- $I_{\max} = 10.0$

This output is not sent directly to analog hardware. Instead, the code splits it into heating and cooling demand:

$$\text{heaterLevel} = \max(0, u)$$

$$\text{coolingRequest} = \max(0, -u)$$

Because the heater hardware is switched through GPIO, the heater command is implemented using time proportioning over a window of length T_w:

$$t_{on} = T_w \cdot \text{heaterLevel}$$

At runtime, the heater is on when the current time within the window is less than t_{on}, and off otherwise. This is the discrete pulse-width-modulated behavior implemented by `TimeProportionedActuator.apply_window()`.

6.4 Fuzzy-PID Temperature Controller

The fuzzy-PID controller in controllers.py is not a completely separate control law. It is a gain-scheduled PID controller that modifies the effective PID gains based on the present temperature error and the error rate.

The same temperature error is used:

$$e(k) = T_{sp}(k) - T(k)$$

and the same error-rate estimate is used:

$$\Delta e(k) = \frac{e(k) - e(k-1)}{\Delta t}$$

The code normalizes these quantities:

$$e_n(k) = \text{clamp}\left(\frac{e(k)}{E_s}, -1, 1\right)$$

$$\Delta e_n(k) = \text{clamp}\left(\frac{\Delta e(k)}{\Delta E_s}, -1, 1\right)$$

where the default scaling constants are:

- $E_s = 5.0$
- $\Delta E_s = 2.0$

The controller then forms:

$$m = |e_n(k)|$$

$$t = |\Delta e_n(k)|$$

and computes gain multipliers:

$$f_{K_p} = \text{clamp}(1 + 0.7m,\ 0.5,\ 2.0)$$

$$f_{K_i} = \text{clamp}(1 - 0.5t + \delta_i,\ 0.2,\ 1.5)$$

$$f_{K_d} = \text{clamp}(1 + 0.8t,\ 0.5,\ 2.0)$$

where:

$$\delta_i = 0.2 \text{ if } e_n(k)\Delta e_n(k) < 0$$

$$\delta_i = 0 \text{ otherwise}$$

This means:

- K_p increases when the temperature error magnitude is large.

- K_d increases when the error is changing rapidly.
- K_i is reduced when the response is changing quickly, but it receives a small boost when the error and its rate have opposite signs, which indicates motion back toward the setpoint.

The effective gains become:

$$K_p^* = K_{p0} f_{K_p}$$

$$K_i^* = K_{i0} f_{K_i}$$

$$K_d^* = K_{d0} f_{K_d}$$

and the controller output is then evaluated by the same PID form:

$$u_f(k) = \text{clamp}\left(K_p^* e(k) + K_i^* I(k) + K_d^* D(k),\ -u_{\max},\ u_{\max}\right)$$

with the same integral state, derivative estimate, and output saturation structure as the standard PID controller.

Therefore, the implemented fuzzy-PID is best described as a fuzzy gain adaptation layer wrapped around the same bounded PID structure.

6.5 Ventilation, Humidity, and Temperature Interaction

The system couples temperature and humidity through the fan. The fan serves two purposes:

- cooling when the chamber is too warm
- dehumidification / ventilation when relative humidity is too high

This interaction is implemented in `ClimateController.compute_outputs()` in climate_control.py.

Temperature-to-fan interaction Temperature-based cooling is controlled by hysteresis:

$$T_{on} = T_{sp} + \Delta T_{on}$$

$$T_{off} = T_{sp} + \Delta T_{off}$$

where the defaults are:

- $\Delta T_{on} = 0.5^\circ C$

- $\Delta T_{off} = 0.0^{\circ}C$

The cooling state is updated as:

- if $T \geq T_{on}$, cooling becomes active
- if $T \leq T_{off}$, cooling becomes inactive
- otherwise, the previous cooling state is retained

When temperature-based cooling is active:

$$\text{fanLevel} = 1$$

and the heater is forced off:

$$\text{heaterLevel} = 0$$

This prevents the heater and fan from fighting each other.

Humidity-to-fan interaction The humidity controller is threshold based with hysteresis:

$$RH_{low} = RH_{th} - \frac{h}{2}$$

$$RH_{high} = RH_{th} + \frac{h}{2}$$

where:

- RH_{th} is the humidity threshold
- h is the hysteresis width

The code applies:

- if $RH > RH_{high}$, ventilation is activated
- if $RH < RH_{low}$, ventilation is deactivated
- otherwise, the previous ventilation state is retained

When ventilation is active, the fan command is forced on:

$$\text{fanLevel} = \max(\text{fanLevel}, 1) = 1$$

Combined interaction The final fan command is therefore controlled by the logical OR of:

- temperature cooling request
- humidity ventilation request

In words:

- high temperature can turn the fan on
- high humidity can also turn the fan on
- either condition is sufficient to run the fan at full power

This matters physically because ventilation tends to reduce both temperature and humidity. The fan is therefore a shared actuator between the thermal and humidity subsystems.

6.6 Humidifier Disturbance and System Limitation

In the physical setup, the humidifier was not implemented as a controllable actuator. Although power could be switched externally, the device still required a manual press of a button on its internal circuit board before it would begin operating. Because of this hardware limitation, the humidifier could not be commanded from the Raspberry Pi in the same way as the heater and fan.

As a result, the humidifier acted as a disturbance rather than a controller. Once manually started, it remained on during the experiment and continuously added moisture to the chamber. The embedded system did not regulate humidity addition directly. Instead, the code handled the humidity increase indirectly:

- the humidifier increased relative humidity as an external disturbance
- the humidity threshold logic detected excessive RH
- the fan was used to ventilate and reduce RH

This is an important practical limitation to mention because it explains why the humidity path in software is supervisory and threshold-based rather than a full closed-loop humidity addition controller.

6.7 Data Logging and Visualization

The control loop logs one row per sample when CSV logging is enabled. The logged signals are:

- `timestamp`
- `temperature`
- `relative_humidity`
- `heater_command`
- `fan_command`

The code also supports a real-time single-window plot that displays:

- temperature

- relative humidity
- heater command
- fan command

This plotting feature is useful for live monitoring, but the report should ultimately present analysis figures derived from logged data rather than screenshots alone.

7. Results

The controller performance was evaluated using the logged CSV data generated during the physical chamber experiments. In both cases, the temperature setpoint was `23.0 C` and the humidity threshold was `65.0 %`. Because the humidifier remained on as a disturbance source, the logged responses reflect coupled temperature-humidity behavior rather than an isolated temperature-only test.

7.1 PID Results

The PID-controlled run started at approximately `19.6 C`, below the target setpoint, while the humidifier disturbance caused the relative humidity to rise quickly and activate ventilation. The figure below shows the full measured response of temperature, relative humidity, heater command, and fan command.

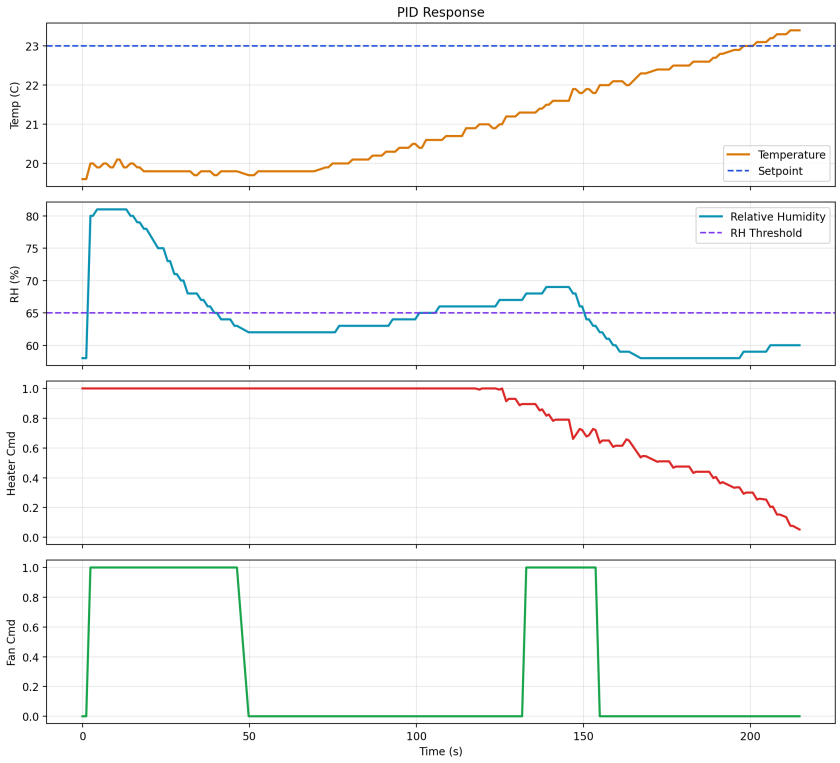

Figure 2: PID experimental response from the logged CSV data.

The PID controller produced the following measured performance:

- Rise time (10-90%): `186.5 s`
- Settling time within `±0.5 C`: `176.9 s`
- Peak temperature: `23.4 C`
- Overshoot: `0.4 C`
- Steady-state error: `0.17 C`
- Maximum recorded RH: `81.0 %`

Overall, the PID controller produced a relatively slow but controlled temperature rise. Overshoot remained small, and the final temperature stayed close to the setpoint. The heater command remained high for much of the run, indicating that the controller was working against both the initial temperature deficit and the cooling/dehumidifying effect of fan operation.

7.2 Fuzzy-PID Results

The fuzzy-PID-controlled run started closer to the setpoint at approximately `21.7 C`. Its initial response was more aggressive than the standard PID case,

but fan activity associated with both cooling and humidity control produced a stronger coupled disturbance during the experiment.

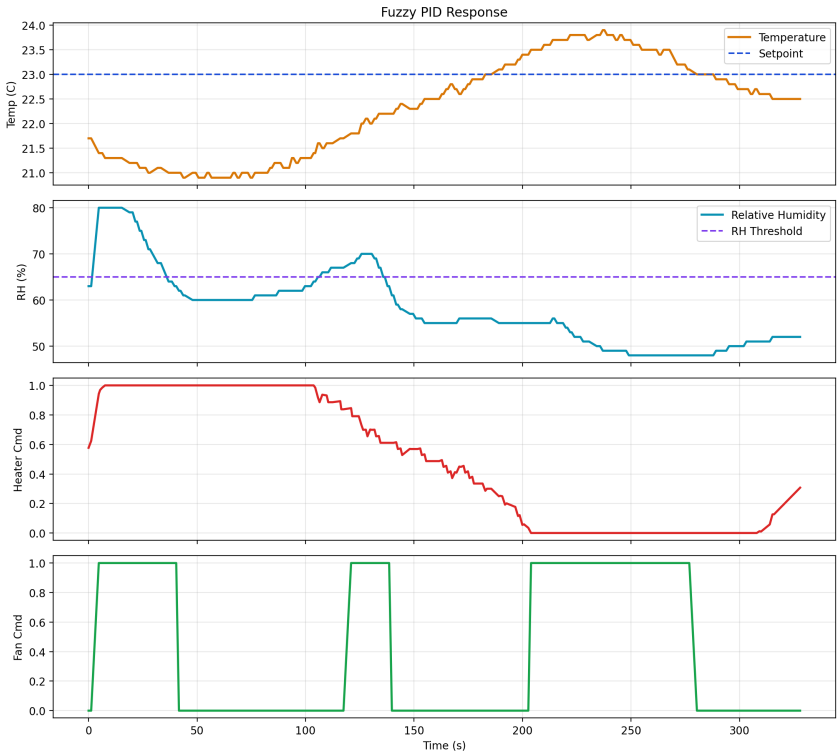

Figure 3: Fuzzy-PID experimental response from the logged CSV data.

The fuzzy-PID controller produced the following measured performance:

- Rise time (10-90%): `51.1 s`
- Settling time within `±0.5 C`: `255.0 s`
- Peak temperature: `23.9 C`
- Overshoot: `0.9 C`
- Steady-state error: `-0.47 C`
- Maximum recorded RH: `80.0 %`

Compared with the standard PID controller, the fuzzy-PID reached the neighborhood of the setpoint much more quickly. However, it also exhibited greater overshoot and a longer final settling time. The lower average heater demand and higher average fan activity indicate that this run experienced stronger interaction between heating and ventilation, which made the final temperature regulation less consistent.

7.3 Logged Data Summary

The combined comparison figure below provides a direct visual comparison of the two experimental runs.

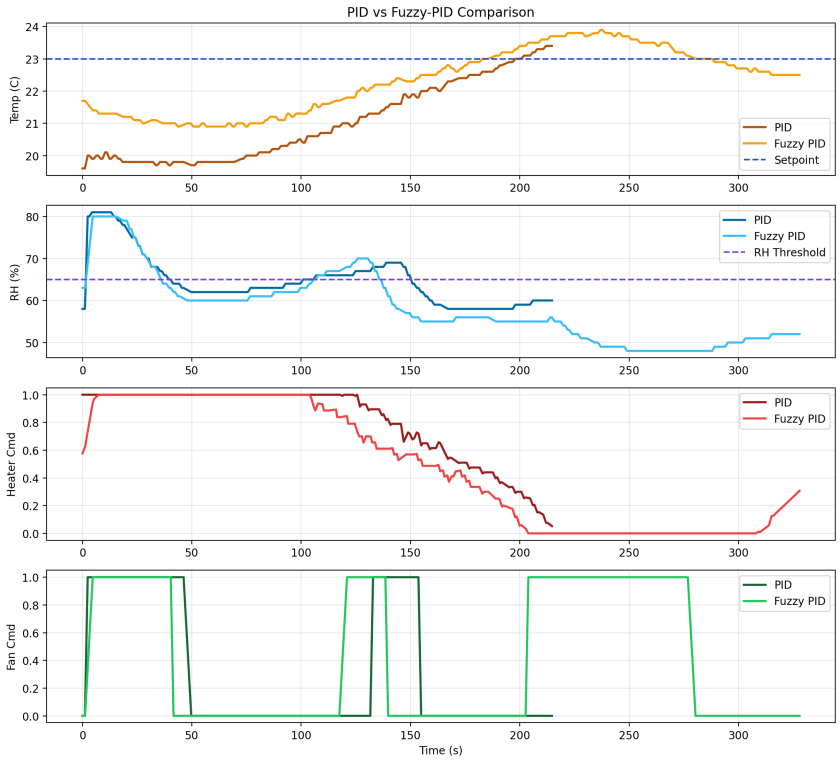

Figure 4: Direct comparison of PID and fuzzy-PID experimental runs.

The quantitative summary derived from the CSV logs is shown below.

Controller	Setpoint	Settling Time	Overshoot	SS Error	Notes
PID	23.0 C	176.9 s	0.40 C	0.17 C	Lower overshoot and tighter final regulation, but with higher mean heater demand throughout the run.
Fuzzy PID	23.0 C	255.0 s	0.90 C	-0.47 C	Faster initial rise, but more fan activity, greater overshoot, and a lower final temperature by the end of the run.

Table 2: Summary metrics derived from the experimental CSV logs.

8. Controller Comparison

The two controllers exhibited a clear tradeoff between speed and stability. The fuzzy-PID controller responded much faster at the beginning of the run, as shown by its shorter `10-90%` rise time. This indicates that the adaptive-gain logic made the controller more aggressive when the chamber temperature was far from the setpoint.

That faster response, however, came with a performance penalty. The fuzzy-PID run showed larger overshoot, a longer settling time, and a negative steady-state error by the end of the experiment. In practical terms, this means the controller approached the setpoint quickly but did not maintain the same final thermal stability as the conventional PID controller.

The standard PID controller responded more slowly, but it maintained a tighter final temperature near the setpoint and produced less overshoot. In this experiment, the PID controller behaved more conservatively and gave the more stable final thermal response.

The humidity disturbance is important when interpreting the comparison. Because the humidifier remained on as a disturbance source, both controllers were tested under repeated fan activation for dehumidification. As a result, this was not a pure heater-only temperature regulation test. Instead, it was a coupled climate-control experiment in which the fan simultaneously influenced temperature and humidity. This coupling likely amplified the differences between the two controllers, especially during periods of elevated RH.

9. Simulink Modeling

9.1 Plant Model

For Simulink comparison, the chamber temperature can be modeled as a first-order thermal system with additive heater, fan, humidifier-cooling, and ice-cup disturbance terms. A compact continuous-time temperature model is:

$$\tau_T \frac{dT}{dt} = (T_{amb} - T) + K_h u_h - K_v u_f - K_m d_m - K_i d_i$$

where:

- T is chamber temperature
- T_{amb} is ambient temperature
- u_h is heater command
- u_f is fan command
- d_m is humidifier disturbance
- d_i is ice-cup disturbance
- K_h, K_v, K_m, K_i are lumped gains
- τ_T is the thermal time constant

Ignoring disturbances for transfer-function discussion, the nominal heater-to-temperature plant becomes:

$$G_T(s) = \frac{K_T}{\tau_T s + 1}$$

Relative humidity can be modeled with a parallel first-order disturbance-driven model:

$$\tau_{RH} \frac{dRH}{dt} = (RH_{amb} - RH) + K_{hum} d_m - K_{vent} u_f$$

This model matches the physical interpretation of the implemented code:

- heater raises temperature
- fan removes heat and humidity
- humidifier increases RH as a disturbance
- ice-cup insertion acts as a cooling disturbance

9.2 Simulink Controller Structure and Model Limitations

The Simulink models were built to represent the same overall controller structure used in the embedded implementation. In both the PID and fuzzy-PID cases, the model used the chamber temperature error, bounded controller output, heater actuation, fan-based cooling, and humidity-triggered ventilation

logic. The fuzzy-PID version additionally included gain adaptation based on normalized error magnitude and error-rate magnitude.

Although this structure matched the intended control logic, the time-domain simulation results were very different from the measured chamber behavior. For that reason, the Simulink response plots are not displayed in this report. The controller diagrams are still included because they communicate the conceptual structure of the PID and fuzzy-PID implementations, but the numerical simulation responses were not considered representative enough of the real chamber to support direct comparison.

Several factors likely contributed to the mismatch between simulation and experiment:

- The thermal plant was modeled as a simplified first-order system, while the real chamber likely had more complex heat storage, nonuniform mixing, and leakage behavior.
- The foam cooler box introduced spatial temperature variation and localized heat retention that were not captured well by a low-order lumped model.
- The fan was a shared actuator for both cooling and dehumidification, which created strong coupling between the temperature and humidity dynamics.
- The humidifier acted as a persistent disturbance rather than as a controllable actuator, making the physical system less ideal and less repeatable than the simulation assumptions.
- The DHT11 sensor introduced coarse and sometimes noisy measurements, especially for humidity, which affected both controller behavior and model-tuning quality.
- The real actuators were implemented through binary switching rather than true analog control, so switching, delay, and transport effects were only approximately represented in simulation.

Based on the observed experiments, the strongest sources of disagreement were likely the oversimplified plant model, the strong temperature-humidity coupling created by ventilation, and the uncontrolled humidifier disturbance.

9.3 Simulink Figures

The Simulink controller diagrams used for the report are shown below.

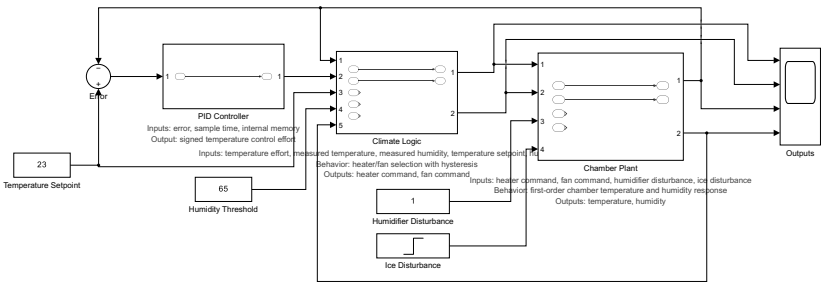

Figure 5: Conceptual PID Simulink controller diagram.

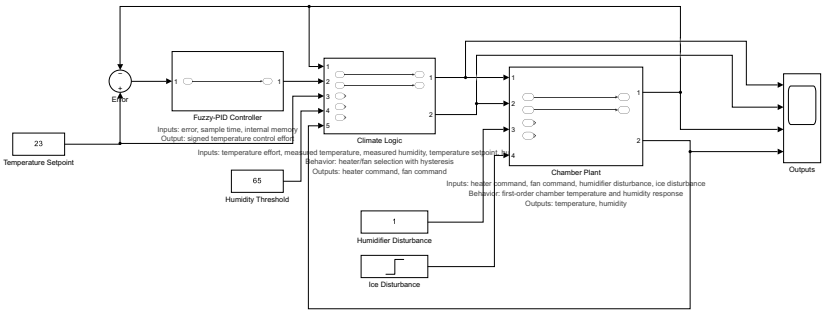

Figure 6: Conceptual fuzzy-PID Simulink controller diagram.

These diagrams summarize the controller structure used for the conceptual comparison between the conventional PID and fuzzy-PID approaches.

10. Discussion

The embedded controller successfully implemented automatic chamber regulation using a Raspberry Pi, a DHT11 sensor, and GPIO-driven actuators. The code was able to log data at approximately `1 Hz`, drive the heater through time-proportioned control, and activate ventilation in response to both temperature and relative humidity conditions. This provided a workable experimental platform for comparing PID and fuzzy-PID temperature control under realistic laboratory disturbances.

The results also highlighted several important practical limitations. The DHT11 sensor provides relatively coarse and sometimes noisy measurements, particularly for humidity, which makes precise performance evaluation more difficult than with a higher-grade sensor. In addition, the fan hardware could not be operated reliably at intermediate power levels, so it had to be treated as a binary actuator. This limited the cooling and dehumidification path to full `ON/OFF` behavior rather than smoothly modulated ventilation.

The most important experimental limitation was the humidifier hardware. Even when electrical power was applied through the external switching stage, the de-

vice still required a manual button press on its onboard circuit board before it would begin operating. As a result, the humidifier could not be used as a true closed-loop humidity actuator and instead behaved as a continuous disturbance source throughout the experiments. This meant that the embedded controller responded to humidity primarily through ventilation rather than through coordinated humidity addition and removal.

Because of these hardware constraints, the measured controller behavior should be interpreted as the response of a coupled temperature-humidity system rather than an ideal single-input thermal system. The fan was shared between cooling and dehumidification, and this coupling strongly affected the final controller comparison. The PID controller appeared more robust under this coupled disturbance environment, while the fuzzy-PID controller achieved faster initial heating but was more sensitive to the interaction between heating and ventilation.

11. Conclusion

This project developed an embedded environmental control system for a small insulated chamber built from a foam cooler box and controlled with a Raspberry Pi. The system successfully measured chamber temperature and relative humidity, applied heater and fan actuation through GPIO-driven hardware, logged the data to CSV, and allowed comparison between PID and fuzzy-PID temperature control strategies.

Experimental results showed that the fuzzy-PID controller produced a faster initial response, but the standard PID controller gave the more stable overall temperature regulation under the coupled temperature-humidity disturbance environment. In particular, the PID controller had lower overshoot and a smaller final steady-state error, while the fuzzy-PID controller responded more aggressively but was more sensitive to the fan and humidity interaction.

The experiments also highlighted an important hardware limitation: the humidifier could not be directly controlled by the embedded system and therefore behaved as a disturbance source rather than as a true humidity actuator. Even with that limitation, the project demonstrated effective closed-loop environmental control and provided a useful comparison of two controller designs under realistic laboratory conditions.

12. Appendix A: Code

The full codebase for this project is available in the local project folder and in the project repository:

- control_systems_final_project repository

Only the most important controller-related code fragments are included below to ensure the appendix remains succinct.

A.1 PID Controller Update

The standard PID controller computes the temperature error, updates the bounded integral term, evaluates the derivative from the previous error, and saturates the final output to the range `[-1, 1]`.

```
def update(self, setpoint: float, measurement: float, dt: float) -> float:
    error = setpoint - measurement
    if dt <= 0.0:
        dt = 1e-6

    self._integral += error * dt
    self._integral = clamp(
        self._integral,
        -self.integral_limit,
        self.integral_limit,
    )

    derivative = 0.0
    if self._has_previous_error:
        derivative = (error - self._previous_error) / dt

    output = (
        self.kp * error
        + self.ki * self._integral
        + self.kd * derivative
    )
    output = clamp(output, -self.output_limit, self.output_limit)

    self._previous_error = error
    self._has_previous_error = True
    return output
```

A.2 Fuzzy-PID Gain Adaptation

The fuzzy-PID implementation uses the same PID structure but modifies the effective gains online according to normalized error and error-rate values.

```
def update(self, setpoint: float, measurement: float, dt: float) -> float:
    error = setpoint - measurement
    delta_error = 0.0 if not self._has_previous_error else (error - self._previous_error) /
    kp_factor, ki_factor, kd_factor = self._infer_gain_adjustments(error, delta_error)

    original = (self.kp, self.ki, self.kd)
    self.kp = self.base_kp * kp_factor
    self.ki = self.base_ki * ki_factor
    self.kd = self.base_kd * kd_factor
```

```
    output = super().update(setpoint, measurement, dt)
    self.kp, self.ki, self.kd = original
    return output
```

A.3 Climate Logic Mapping

The controller output is translated into heater and fan commands through temperature hysteresis and humidity-threshold ventilation logic.

```
heater_level = max(0.0, temp_effort)
if temperature_cooling_active:
    heater_level = 0.0

fan_level = 1.0 if temperature_cooling_active else 0.0
if ventilation_active:
    fan_level = max(fan_level, 1.0)

return ControlOutputs(
    heater_level=heater_level,
    fan_level=fan_level,
    ventilation_active=ventilation_active,
)
```